

CPSC 8440: Generative AI: Technical and Human Perspectives

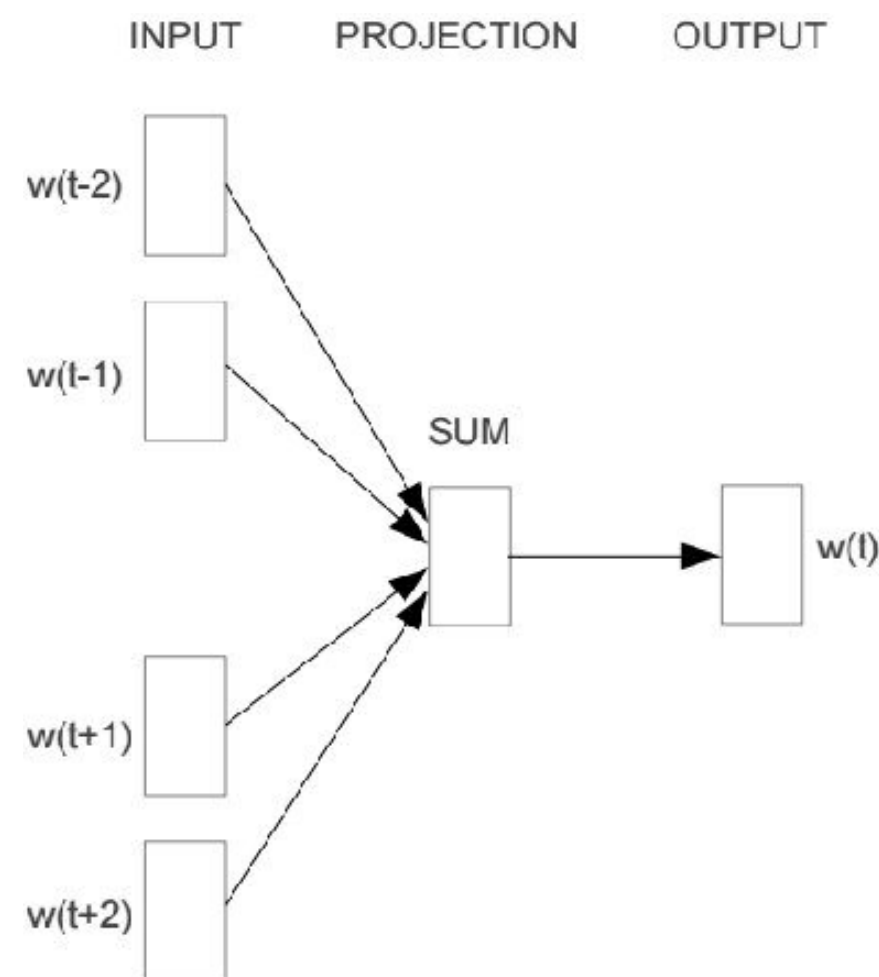
Module 1.3.2: Word2Vec

Word2Vec

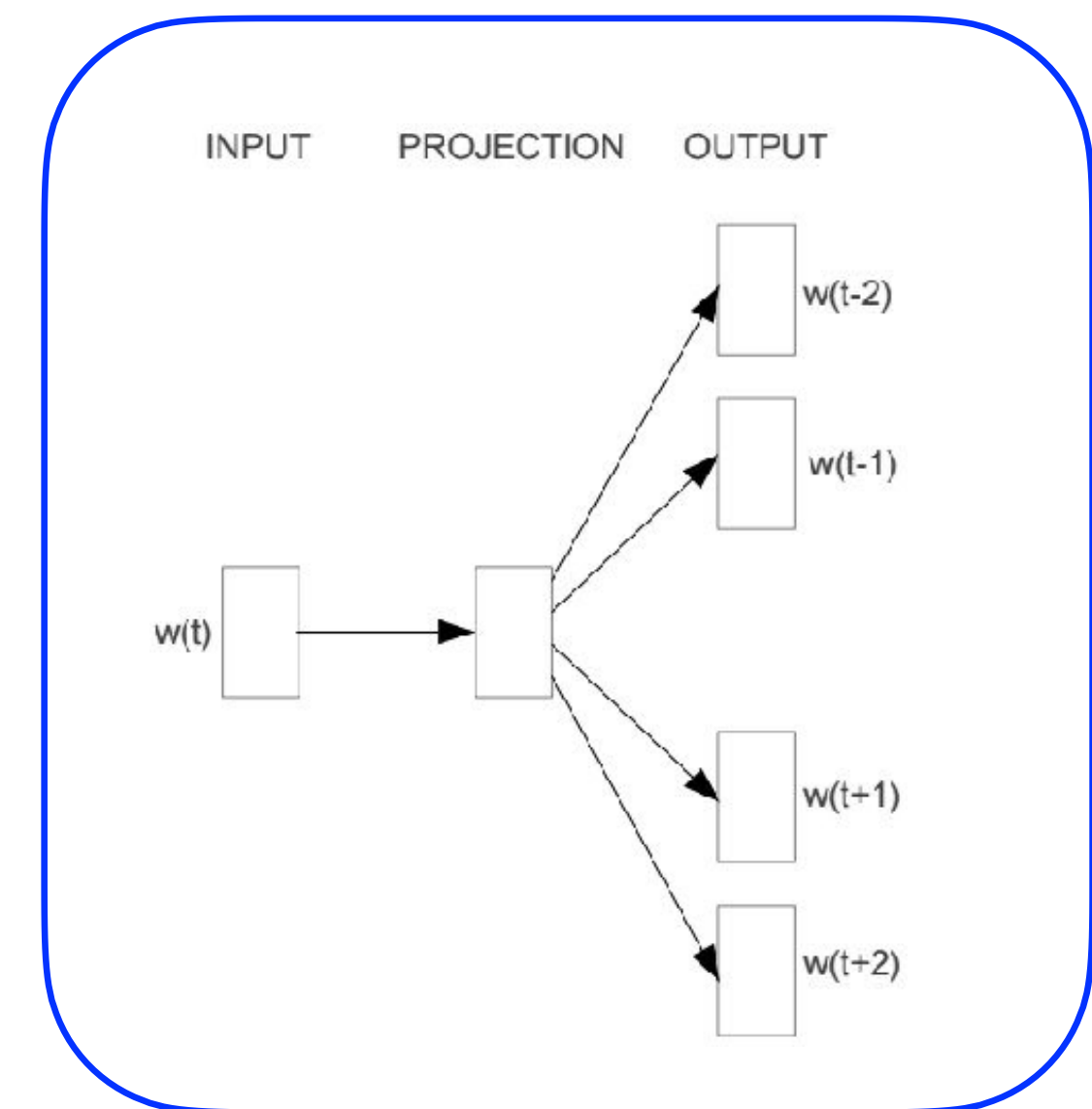
- (Mikolov et al 2013a): Efficient Estimation of Word Representations in Vector Space
- (Mikolov et al 2013b): Distributed Representations of Words and Phrases and their Compositionality



Tomáš Mikolov



Continuous Bag of Words (CBOW)

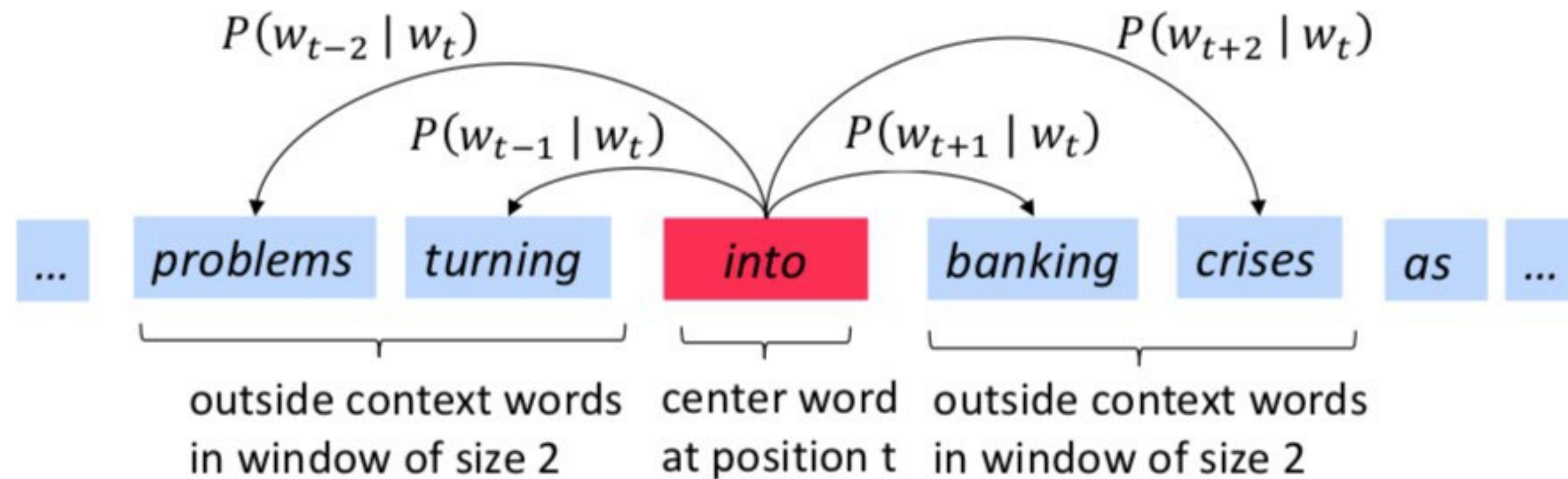


Skip-gram

Skip-gram

- Assume that we have a large corpus $w_1, w_2, \dots, w_T \in V$
- Key idea: Use each word to predict other words in its context
- Context: a fixed window of size $2m$ ($m = 2$ in the example)

A classification problem!

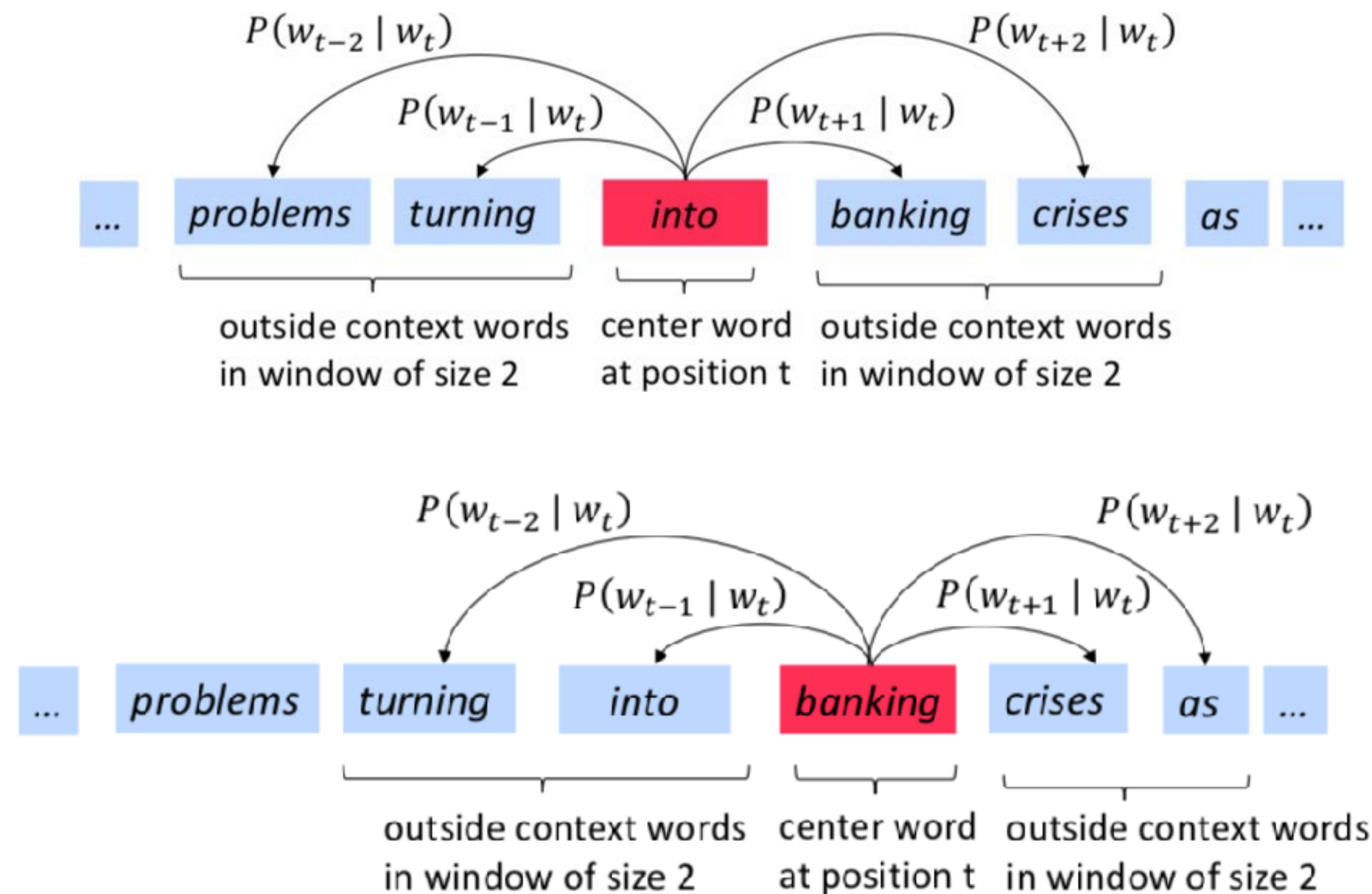


$P(b | a)$ = given the center word is a , what is the probability that b is a context word?

$P(\cdot | a)$ is a probability distribution defined over V :

$$\sum_{w \in V} P(w | a) = 1$$

Skip-gram



Convert into training data:

(into, problems)

(into, turning)

(into, banking)

(into, crises)

(banking, turning)

(banking, into)

(banking, crises)

(banking, as)

...

Our goal is to find parameters that can maximize

$P(\text{problems} | \text{into}) \times P(\text{turning} | \text{into}) \times P(\text{banking} | \text{into}) \times P(\text{crises} | \text{into}) \times$

$P(\text{turning} | \text{banking}) \times P(\text{into} | \text{banking}) \times P(\text{crises} | \text{banking}) \times P(\text{as} | \text{banking}) \dots$

Skip-gram: objective function

- For each position $t = 1, 2, \dots, T$, predict context words within context size m , given center word

w_t :

$$\mathcal{L}(\theta) = \prod_{t=1}^T \prod_{-m \leq j \leq m, j \neq 0} P(w_{t+j} \mid w_t; \theta)$$

all the parameters to
be optimized

- It is equivalent to minimizing the (average) negative log likelihood

How to define $P(w_{t+j} | w_t; \theta)$?

- Use two sets of vectors for each word in the vocabulary

$\mathbf{u}_a \in \mathbb{R}^d$: vector for center word a , $\forall a \in V$

$\mathbf{v}_b \in \mathbb{R}^d$: vector for context word b , $\forall b \in V$

- Use inner product $\mathbf{u}_a \cdot \mathbf{v}_b$ to measure how likely word a appears with context word b

$$P(w_{t+j} | w_t) = \frac{\exp(\mathbf{u}_{w_t} \cdot \mathbf{v}_{w_{t+j}})}{\sum_{k \in V} \exp(\mathbf{u}_{w_t} \cdot \mathbf{v}_k)}$$

Does this term
seem familiar?

Since we have to learn both $\mathbf{u}$ and $\mathbf{v}$ together, the training objective is **non-convex**

How many parameters in this model?

How many parameters does this model have (i.e. what is size of θ)?

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log \frac{\exp(\mathbf{u}_{w_t} \cdot \mathbf{v}_{w_{t+j}})}{\sum_{k \in V} \exp(\mathbf{u}_{w_t} \cdot \mathbf{v}_k)}$$

(a) $d|V|$

(b) $2d|V|$

(c) $2m|V|$

(d) $2md|V|$

$V :=$ Vocabulary

$d :=$ dimension of embedding

$m :=$ size of context window

The answer is (b).

Each word has two d -dimensional vectors, so it is $2 \times |V| \times d$.

Word2Vec formulation

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log \frac{\exp(\mathbf{u}_{w_t} \cdot \mathbf{v}_{w_{t+j}})}{\sum_{k \in V} \exp(\mathbf{u}_{w_t} \cdot \mathbf{v}_k)}$$

Q: Why do we need two vectors for each word?

- Because one word is not likely to appear in its own context window, e.g., $P(\text{dog} \mid \text{dog})$ should be low. If we use one set of vectors only, it essentially needs to minimize $\mathbf{u}_{\text{dog}} \cdot \mathbf{u}_{\text{dog}}$

Q: Which set of vectors are used as word embeddings?

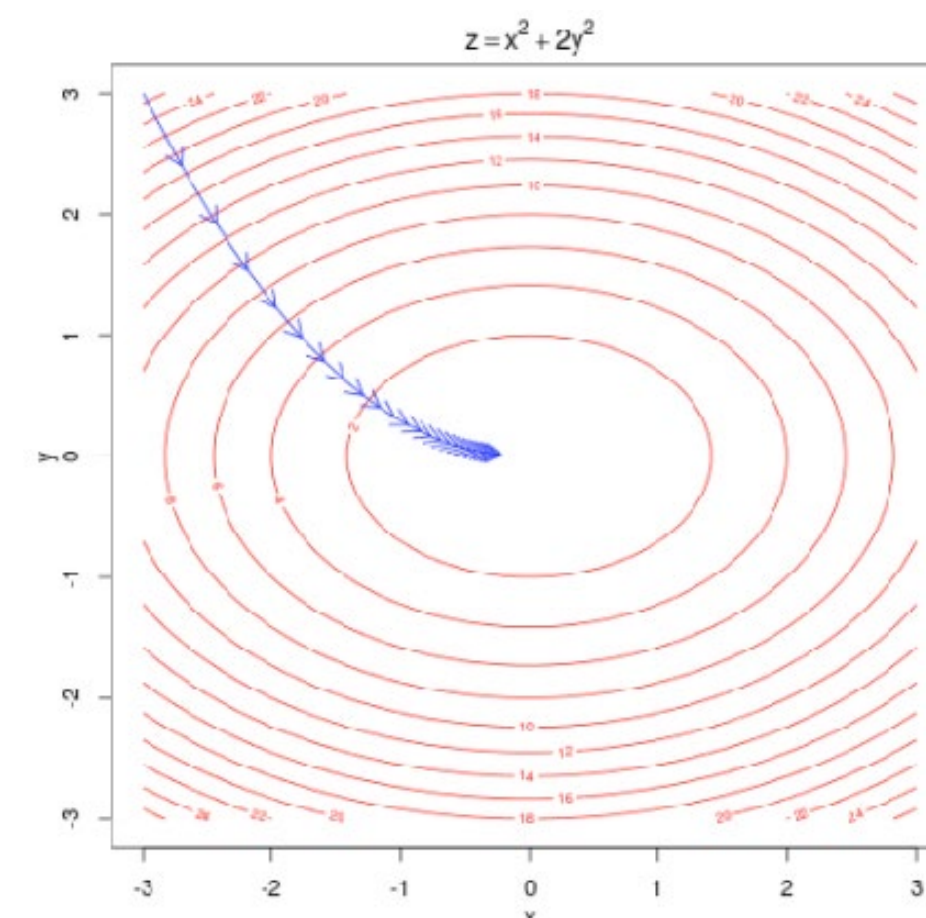
- This is an empirical question. Typically just $\mathbf{u}_w$ but you can also concatenate the two vectors.

How to train this model?

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log \frac{\exp(\mathbf{u}_{w_t} \cdot \mathbf{v}_{w_{t+j}})}{\sum_{k \in V} \exp(\mathbf{u}_{w_t} \cdot \mathbf{v}_k)}$$

- To train such a model, we compute the vector gradient $\nabla_{\theta} J(\theta)$
- θ represents all $2d|V|$ model parameters in one vector.

$$\theta = \begin{bmatrix} v_{aardvark} \\ v_a \\ \vdots \\ v_{zebra} \\ u_{aardvark} \\ u_a \\ \vdots \\ u_{zebra} \end{bmatrix}$$



Overall algorithm

- Input: text corpus, embedding size d , vocabulary V , context size m
- Initialize $\mathbf{u}_i, \mathbf{v}_i$ randomly $\forall i \in V$
- Run through the training corpus and for each training instance (t, c) :

- Update $\mathbf{u}_t \leftarrow \mathbf{u}_t - \eta \frac{\partial y}{\partial \mathbf{u}_t}; \quad \frac{\partial y}{\partial \mathbf{u}_t} = -\mathbf{v}_c + \sum_{k \in V} P(k|t) \mathbf{v}_k$

- Update $\mathbf{v}_k \leftarrow \mathbf{v}_k - \eta \frac{\partial y}{\partial \mathbf{v}_k}, \forall k \in V; \quad \frac{\partial y}{\partial \mathbf{v}_k} = \begin{cases} (P(k|t) - 1) \mathbf{u}_t & k = c \\ P(k|t) \mathbf{u}_t & k \neq c \end{cases}$

Trained word embeddings available

- word2vec: <https://code.google.com/archive/p/word2vec/>
- GloVe: <https://nlp.stanford.edu/projects/glove/>
- FastText: <https://fasttext.cc/>

Download pre-trained word vectors

- Pre-trained word vectors. This data is made available under the [Public Domain Dedication and License](http://www.opendatacommons.org/licenses/pddl/1.0/) v1.0 whose full text can be found at: <http://www.opendatacommons.org/licenses/pddl/1.0/>.
 - [Wikipedia 2014 + Gigaword 5](#) (6B tokens, 400K vocab, uncased, 50d, 100d, 200d, & 300d vectors, 822 MB download): [glove.6B.zip](#)
 - Common Crawl (42B tokens, 1.9M vocab, uncased, 300d vectors, 1.75 GB download): [glove.42B.300d.zip](#)
 - Common Crawl (840B tokens, 2.2M vocab, cased, 300d vectors, 2.03 GB download): [glove.840B.300d.zip](#)
 - Twitter (2B tweets, 27B tokens, 1.2M vocab, uncased, 25d, 50d, 100d, & 200d vectors, 1.42 GB download): [glove.twitter.27B.zip](#)
- Ruby [script](#) for preprocessing Twitter data

Differ in algorithms, text corpora, dimensions, cased/uncased...

Easy to use!

```
from gensim.models import KeyedVectors
# Load vectors directly from the file
model = KeyedVectors.load_word2vec_format('data/GoogleGoogleNews-vectors-negative300.bin', binary=True)
# Access vectors for specific words with a keyed lookup:
vector = model['easy']
```

```
In [17]: model.similarity('straightforward', 'easy')
```

```
Out[17]: 0.5717043285477517
```

```
In [18]: model.similarity('simple', 'impossible')
```

```
Out[18]: 0.29156160264633707
```

```
In [19]: model.most_similar('simple')
```

```
Out[19]: [('straightforward', 0.7460169196128845),
          ('Simple', 0.7108174562454224),
          ('uncomplicated', 0.6297484636306763),
          ('simplest', 0.6171397566795349),
          ('easy', 0.5990299582481384),
          ('fairly_straightforward', 0.5893306732177734),
          ('deceptively_simple', 0.5743066072463989),
          ('simpler', 0.5537199378013611),
          ('simplistic', 0.5516539216041565),
          ('disarmingly_simple', 0.5365327000617981)]
```